



Transfer Learning

Our Team

A

Shivkant Sawarkar



B

Suraj Samal



C

Vikas Vishwakarma



D

Sachin Nandal



What is Transfer Learning?

- A machine learning technique.
- A model trained on one task is reused for another related task.
- Instead of training a new model from scratch.

For example:

1. A model trained to recognize millions of general images (cars, animals, people, etc.) can be adapted to detect pneumonia from X-ray images.
2. A language model trained on huge text data can be fine-tuned for sentiment analysis..

Intro Transfer Learning Rationale

In practice, very few people train an entire Convolutional Network from scratch (i.e. random initialization of weights), because it is relatively rare to have a dataset of sufficient size.

Instead, it is common to pretrain a ConvNet on a very large dataset (e.g. ImageNet, which contains 1.2 million images with 1000 categories), and then use the ConvNet either as an initialization or a fixed feature extractor for the task of interest. The three major Transfer Learning scenarios look as follows:



Training extensive models on vast image datasets like ImageNet can sometimes take weeks.

A model trained on so much data would have useful embeddings that can be applied to other image domains e.g. edge detectors, pattern and blob detectors.

Example, an ImageNet model trained to detect Tigers, Lions and Horses might be useful to detect other 4-legged mammals.

How it works

Usually NN consist of two parts,:

1. CNN encoder is used to extract important features from an image. And encode them in lower dimension linear vector.
2. Then feed this linear feature vector with quality information to a linear classifier for classification.

For CNN encoder to get good feature understanding we first train it on the large dataset with many examples, with that much data it learn to extract features effectively.

Then next time we have a small datasets we use trained feature extractor and train the linear layer for the classification task. This helps up for good accuracy on the small dataset without overfitting issues.

This transfer of learning from large datasets to small datasets is the Transfer Learning.

Why was Transfer Learning Introduced?

1

Requires Massive Data

Training a robust vision or language model from scratch requires hundreds of thousands—if not millions—of labeled examples.

2

High Computational Cost

Training large models from scratch requires massive GPU/TPU clusters running for days or weeks. Impossible for startups, researchers.

3

Slow Time-to-Market

The time spent collecting data, labeling it, and training the model from scratch creates a massive bottleneck, delaying deployment.

4

The ‘Cold-Start’ Problem

If you don't have historical data for a new application, a traditional model cannot function effectively, leading to poor initial performance.

Transfer Learning Formula:

Suppose:

- Transfer learning can also be represented as:

$$\mathbf{D}_s, \mathbf{T}_s \rightarrow \mathbf{D}_T, \mathbf{T}_T$$

where:

- $\mathbf{D}_s$ = source domain (e.g., ImageNet)
- $\mathbf{T}_s$ = source task
- $\mathbf{D}_T$ = target domain
- $\mathbf{T}_T$ = target task.

The goal is:

$$\mathbf{P}(\mathbf{Y}_T | \mathbf{X}_T) \mathbf{P}(\mathbf{Y}_T | \mathbf{X}_T)$$

using knowledge learned from:

$$\mathbf{P}(\mathbf{Y}_s | \mathbf{X}_s) \mathbf{P}(\mathbf{Y}_s | \mathbf{X}_s)$$

1. Fixed Feature Extractor

Desc:

1. Use a model already trained on a large dataset (like ImageNet).
2. Remove the final classification layer.
3. Freeze all pretrained layers.
4. Extract features from your images and train only a new classifier.

Example

1. Pretrained model: ResNet trained on ImageNet.
2. New task: Classify cats and dogs.

Architecture:

Input Image



Pretrained CNN (Frozen)



Feature Vector



New Classifier



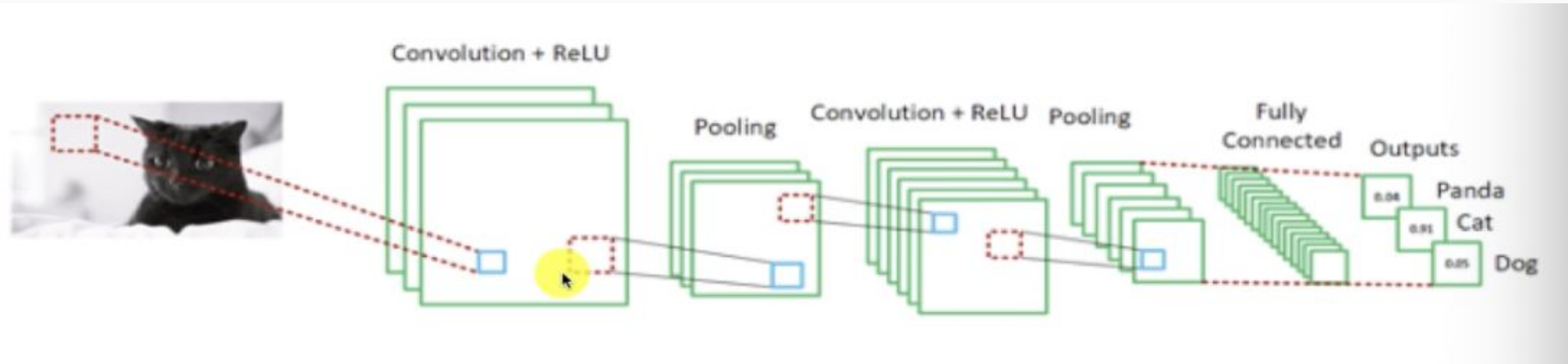
Prediction

The CNN already knows:

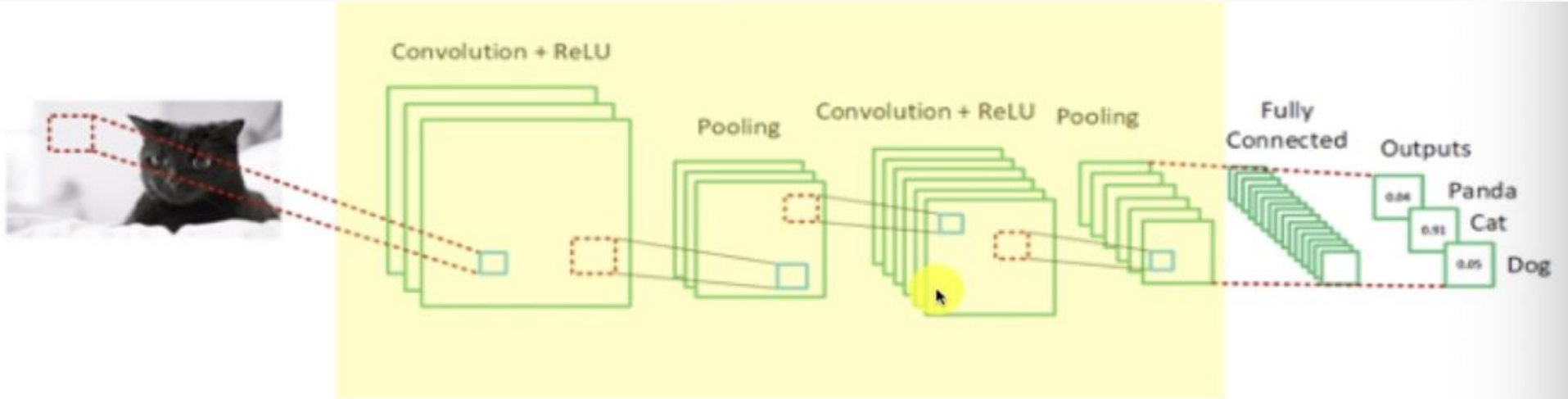
- Edges
- Shapes
- Textures
- Patterns

You only train the final classifier

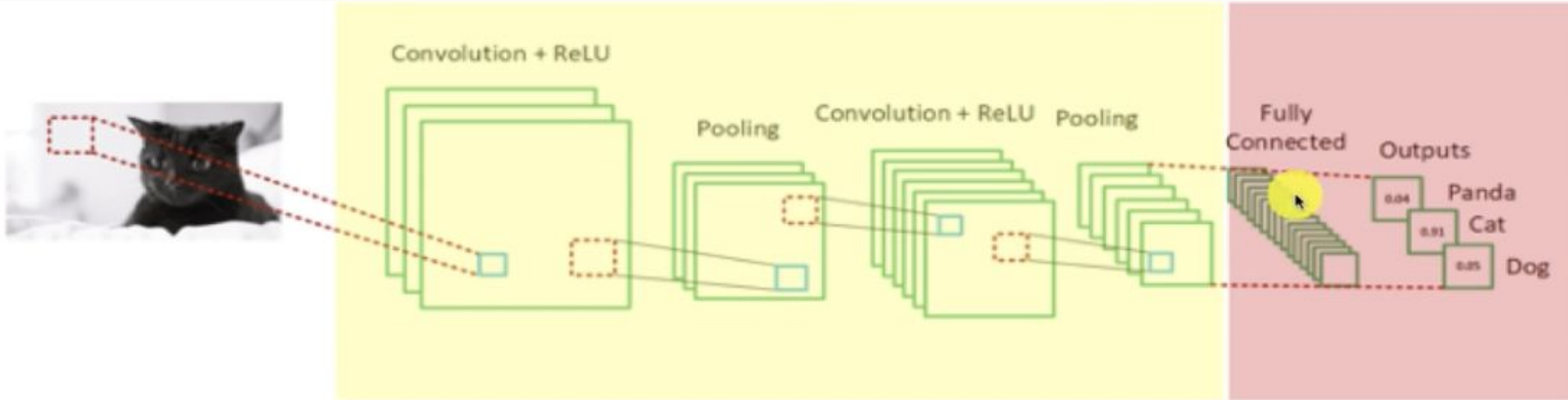
Take a pretrained network:



Freeze the CONV weights/layers - meaning they remain fixed:

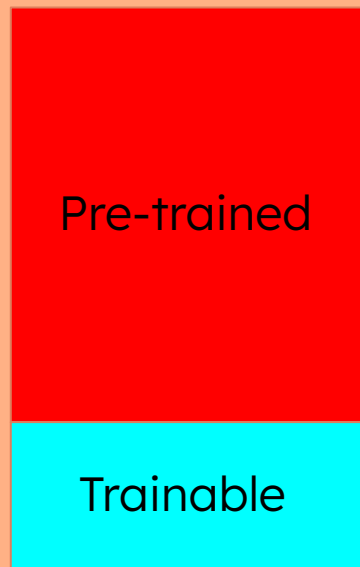


Replace the Top Layer with your Top Layer and train it:



The steps involved in Transfer Learning by Feature Extraction

- **Freeze bottom layers** of a pre-trained network
- **Replace the top half of the network with your top**, so that it outputs only the number of classes in your dataset.



Formula:

Suppose:

- Pretrained model:

$$f(x; \theta_p)$$

where:

- x = input image
- θ_p = pretrained weights (frozen)..

The extracted features are:

$$z = f(x; \theta_p)$$

Then a new classifier is trained:

$$\hat{y} = g(z; \theta_c)$$

Or

$$\hat{y} = g(f(x; \theta_p); \theta_c)$$

where:

- θ_c = parameters of the new classifier.

2. Fine-Tuning a Pretrained Model

Desc:

1. Start with a pretrained model.
2. Unfreeze some or all layers.
3. Continue training on your new dataset

Example

1. ImageNet model → Medical X-ray classification.
2. The model adjusts its weights to learn medical features

Architecture:

Input Image



Pretrained CNN

(Some layers trainable)



Modified Classifier



Prediction

Formula:

Suppose:

- Pretrained model:

$$\hat{y} = g(f(x; \theta_p); \theta_c)$$

where:

- x = input image
- θ_p = pretrained weights (frozen)

However, now both the classifier and some pretrained layers are updated.

In Fine Tuning, we complete all the steps in Feature Extraction but then we

- Unfreeze all or parts of the pre-trained model.
- Train the model for a few epochs; this is where we 'fine tune' the weights of the pre-trained model.

The intuition behind this is that earlier feature maps in a ConvNet learn generic features, while the later layers learn specifics about the image dataset. By fine tuning, we change those specifics from the pre-trained model to the specifics of our dataset.



Transfer Learning Advice:

Learning Rates – use very small learning rates for pre-trained models especially when fine tuning. This is because we know the pretrained weights are already very good and thus don't want to change them too much.

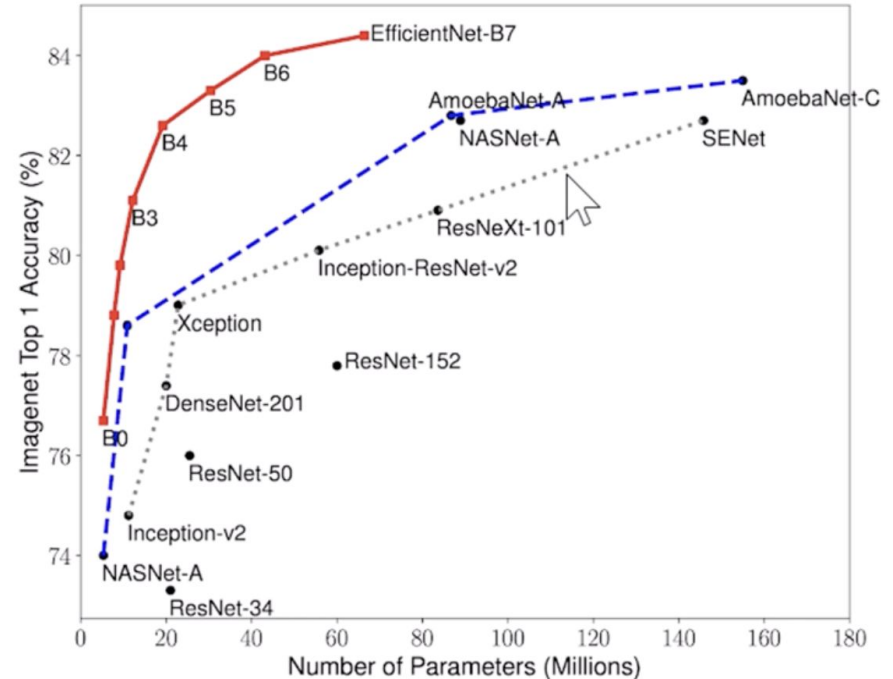
- Pretrained layers: $\eta = 0.00001$
- New classifier: $\eta = 0.001$

Due to parameter sharing, you can train a pre-trained network on images of different sizes.

- Pretrained model:
 - a. 224×224 images
- Fine-tuning:
 - a. 128×128 images
 - b. 256×256 images
 - c. 512×512 images

Model efficiency

1. **Blue Line & Black Lines:**
 - a. Pretrained Model Trained on large datasets.
 - b. Eg:
 - i. ResNet-50
 - ii. Xception
2. **Red Line:**
 - a. After transfer learning & Fine tuning:
 - b. Eg:
 - i. EfficientNet-B7



1. Pretrained ResNet → Detect brain tumors from MRI scans

Pretraining:

1. Dataset: ImageNet contains:
 - a. 1.2 million images
 - b. 1000 classes
2. Input image size:
 - a. $224 \times 224 \times 3$
3. Forward Propagation
4. Output layer.
 - a. 1000 neurons

Transfer Learning

1. 5000 MRI image & Classes: (Tumor, No Tumor)
2. Load Pretrained Model and remove last layer.

Architecture:

MRI Image
↓
ResNet Layers
↓
2048 Features
↓
FC Layer
↓
2 Classes

Feature Extraction:

Freeze all layers

Train only: FC(2048 → 2)

Fine Tuning:

Layer4

Train only: FC(2048 → 2)

1. Pretrained ResNet → Detect brain tumors from MRI scans

Comparison:

Metric	From Scratch	Transfer Learning
Dataset Size	100,000	5,000
Epochs	100	10
Training Time	2 weeks	2 hours
Accuracy	85%	95%

Data Efficiency: Less Data => $100000/5000 = 20x$

Training Speed: Faster => $336/2 = 168x$

Epoch Efficiency: Faster => $100/10 = 10x$

Accuracy Improvement: Higher => $95 - 85 = 10\%$

Transfer Learning Example: I

1. Pretrained ResNet-50 → Self Driving Model

Pretraining:

1. Dataset: MS COCO contains:
 - a. 1.2 million images
 - b. Classes: Cars, People, Dogs, Roads, Traffic lights, Bicycles, Buildings
2. Input image size:
 - a. $224 \times 224 \times 3$
3. Forward Propagation
4. Output layer.
 - a. 1000 neurons

Transfer Learning

1. 10000 Road image & Classes: (6 classes)
2. Load Pretrained Model and remove last layer.

Architecture:

Road Image
↓
Pretrained ResNet
↓
Feature Pyramid
Network
↓
Region Proposal Network
↓
Detection Head
↓
Bounding Boxes + Labels

Feature Extraction:

Freeze backbone

Train only: Detection Head

Fine Tuning:

Layer4

FPN

Detection head

Transfer Learning Example: I

1. Pretrained ResNet-50 → Self Driving Model

Comparison:

Metric	From Scratch	Transfer Learning
Dataset Size	10,000,000	100,000
Epochs	100	10
Training Time	Weeks	Days
GPU Cost	Very High	Much Lower

Data Efficiency: Less Data => $10000000/100000 = 100x$

Training Speed: Faster => $100/10 = 10x$

Epoch Efficiency: Faster => $100/10 = 10x$

Efficiency Gain: Higher => $95 - 85 = 10\%$

1. Pretrained EfficientNet-B0 → Detect diseases in tomato leaves Model

Pretraining:

1. Dataset: MS COCO contains:
 - a. 1.2 million images
 - b. Classes
2. Input image size:
 - a. $224 \times 224 \times 3$
3. Forward Propagation
4. Output layer.
 - a. 1000 neurons

Transfer Learning

1. 10000 Road image & Classes: Healthy, Early Blight Late Blight, Leaf Mold, Septoria Leaf Spot
2. Load Pretrained Model and remove last layer.

Architecture:

Leaf Image
↓
EfficientNet
↓
1280 Features
↓
New Classifier
↓
5 Disease Classes

Feature Extraction:

Freeze backbone

Train only: Detection Head

Fine Tuning:

Layer4

FPN

Detection head

1. Pretrained ResNet → Detect brain tumors from MRI scans

Comparison:

Metric	From Scratch	Transfer Learning
Dataset Size	10,000,000	100,000
Epochs	100	10
Training Time	Weeks	Days
GPU Cost	Very High	Much Lower

Data Efficiency: Less Data => $10000000/100000 = 100x$

Training Speed: Faster => $100/10 = 10x$

Epoch Efficiency: Faster => $100/10 = 10x$

Efficiency Gain: Higher => $95 - 85 = 10\%$



Q&A